

运算符与表达式

北京理工大学计算机学院
金旭亮



Java算术运算符

- 在许多程序中都用到了算术运算
- Java算术运算符与C基本一致，看看下面的例子就清楚了：
 - * 乘
 - / 除
 - +, - 加减
 - Java没有提供指数运算符，而是使用库函数`Math.pow()`实现
 - 整除： $7 / 5 = 1$ （如果是浮点数： $7.0 / 5 = 1.4$ ）
 - 取模： $7 \% 5 = 2$

算术运算符的优先级

一些算术运算优先于其它（例如，先乘除后加减），因此，必要时可使用括号保证优先级别。

示例：计算 a, b 和 c 的平均值

- 错： $a + b + c / 3$
- 对： $(a + b + c) / 3$

```
int a = 1, b = 2, c = 3;  
//计算平均值，应该采用浮点数除法，  
//否则，分子分母都是整数，就变成整除了  
double average = ( a + b + c ) / (double) 3;  
System.out.println(average); //2.0
```

注意示例代码中如何将整数数值转换为浮点数。

复合赋值运算符

复合赋值运算符	示例	解释	结果
假设: int a=1, b=2;			
+=	a += 1	a = a + 1	a=2
-=	a -= 1	a = a - 1	a=0
*=	a *= 2	a = a * 2	a=2
/=	a /= b	a = a / b	a=0 (整数, 商为0)
%=	b %= 3	b = b % 3	b=2 (2除3取余数)

```
//复合赋值运算符示例 (注意前一句执行的结果会影响到后一句)
System.out.println(a += 1); //2
System.out.println(a -= 1); //1
System.out.println(a *= 2); //2
System.out.println(a /= b); //1
System.out.println(b % 3); //2
```

示例执行的结果与上表不同，是因为前一条语句修改了变量的值

自增自减运算符

```
int value = 1;  
value++; // 等价于 value += 1;  
System.out.println(value); //2
```

类似地

```
int value = 1;  
value--; // 等价于 value -= 1;  
System.out.println(value); //0
```

逻辑关系运算符

运算符	示例	结果	说明
假设： int m=1, n=2			
== (判等)	m==n	false	值相等，结果为true
!= (不等于)	m!=n	true	值不等，结果为true
>, <, >=, <= (大小比较)	m>n	false	m的值比n大，结果为true，否则，结果为false
&& (与)	m>n && m<100	false	两边式子均为true，结果才为true
(或)	m>n m<100	true	两边只要有一个式子为true，结果就为true
! (非)	!(m>n)	true	括号内的式子为true，结果为false；为false，则结果为true

逻辑关系运算符的示例

```
//逻辑关系运算符
int m = 1, n = 2;
System.out.println(m == n); //false
System.out.println(m != n); //true
System.out.println(m > n); //false
System.out.println(m > n && m < 100); //false
System.out.println(m > n || m < 100); //true
System.out.println(!(m > n)); //true
```



在上述示例代码中，m和n都可以用一个返回int值的表达式代替，从而构建出更复杂的嵌套表达式。



逻辑关系运算符主要用于条件语句中。

位运算符

运算符	示例	结果
&(与)	1 & 0	0
(或)	1 0	1
^(异或)	1 ^ 0	1
~(非)	~0	-1
>>, <<(右移, 左移)	1<<2	4

//位操作运算符

```
System.out.println(1 & 0); //0
System.out.println(1 | 0); //1
System.out.println(1 ^ 0); //1
System.out.println(~0);    //-1
System.out.println(1 << 2); //4
```

注意：使用位运算符时，操作数按二进制表示。

巩固练习：数值的二进制表示与位运算



在《计算机原理》这门课程（或类似课程）中会介绍到使用二进制表示正数和负数的方法。

在科学计算和某些算法中，会用到位运算以提升计算的性能以及降低对计算机系统资源的占用。

请阅读相应教材，或者使用互联网搜索引擎，弄清楚反码、补码跟原码这几个概念，然后编写示例程序，对正数、负数进行各种位操作，观察输出结果，与手工计算的结果进行比对，看看Java中的整数是采用上述哪种码表示的。

运算符的结合性

Precedence of operators (运算符结合性)：除了赋值运算符 `=`，其他所有运算符的结合性都是从左到右。

例如：`x = y = z` 相当于 `x = (y = z)`

```
int x, y, z = 3;  
//实际开发中，别写这种奇葩的代码！  
System.out.println(x = y = z); //3
```



象“`x=y=z`”这样的可能引起误解的代码，在实际开发中应该尽量避免，如果确实需要，也应该采用右边加上括号的形式，或者是添加相应注释，不要卖弄技巧。

从运算符到表达式

使用运算符将变量组合起来，就构成了“**表达式**”。

表达式的运算结果，称为“表达式的值”。

```
int a = 100;  
int b = 10;  
//表达式肯定有一个值，这个值可以被另一个变量所接收  
int result = a / b - 5;  
System.out.println(result); //5  
//可以把表达式当成一个值，传给另外一个函数作为参数  
System.out.println(++a); //101
```

Expression.java



表达式肯定有一个值，因此，你可以把它当成一个数来看待，通过组合表达式构造更复杂的嵌套表达式。
下面来看一个求解方程的例子

示例:

求解方程 $5x^2-4x-1=0$ 的根

$$x = \frac{-b \pm \sqrt{\Delta}}{2a} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

```
//计算一元二次方程的根
private static void CaculateRoot() {
    double a = 5;
    double b = -4;
    double c = -1;
    //依据求根公式，开始构建表达式
    double delta = b * b - 4 * a * c;
    if (delta < 0) {
        System.out.println("此方程无解");
        return;
    }
    double x1 = (-b + Math.sqrt(delta)) / (2 * a);
    double x2 = (-b - Math.sqrt(delta)) / (2 * a);
    System.out.println("x1=" + x1); //x1=1.0
    System.out.println("x2=" + x2); //x2=-0.2
}
```

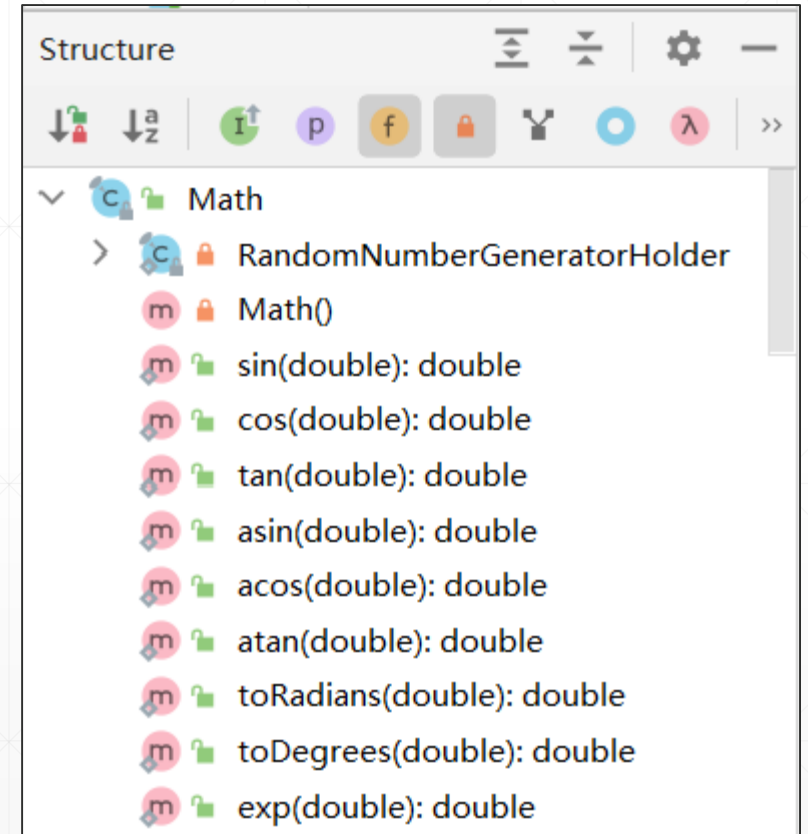
Expression.java

作业



JDK 中有一个Math类，里面提供了一些常用的数学函数。

在IntelliJ的某个Java源文件中输入一个Math，压住Ctrl键，用鼠标点击“Math”这个词，就可以查看到Math类的源码。
请试着读读Math类的源码，读不懂也不要紧，至少你会知道有哪些现成的数学函数可以使用。



下一讲，通过示例学Java基础编程技巧.....